



Localization

V Muthukumar

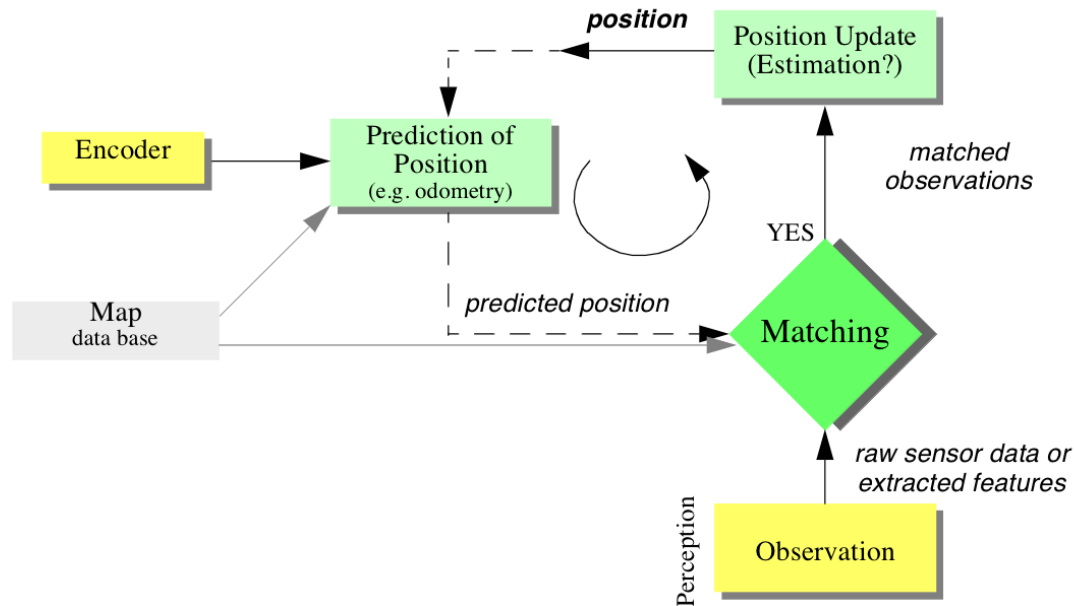
Localization: inferring location given a map





Mobile Robot Localization

- Localization is one of the fundamental problems in mobile robot navigation
- Localization - determine the robot pose from landmarks position and encoder-angular measurements.
- Localization means more than simply determining an absolute pose in space; it means reference/building(?) a map, then identifying the robot's position relative to that map.



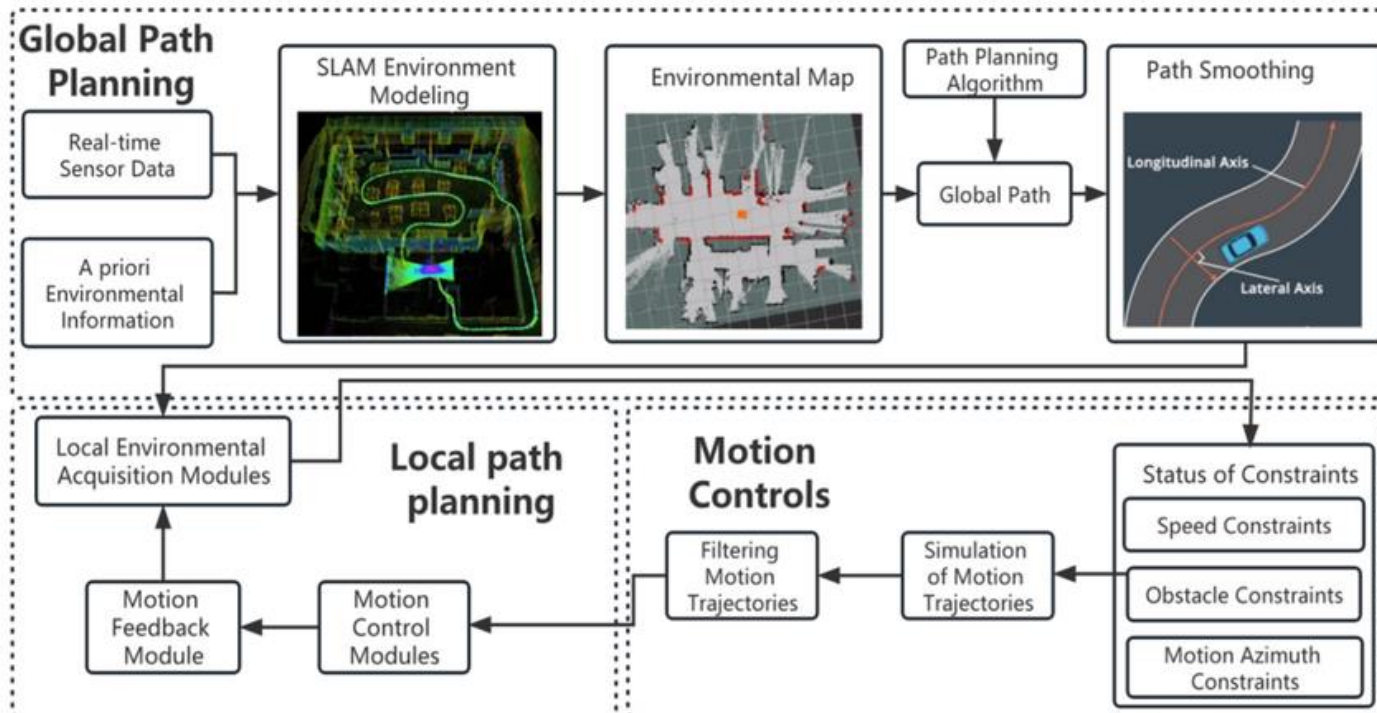
General schematic for mobile robot localization





Navigation

- Robot navigation means the robot's ability to determine its own position in its frame of reference (localization) and then to plan a path towards some goal location.
 - [Global navigation](#), which is the ability to determine one's position in absolute or map-referenced terms, and to move to a desired destination point.
 - [Local navigation](#), the ability to determine one's position relative to objects (stationary or moving) in the environment, and to interact with them correctly.





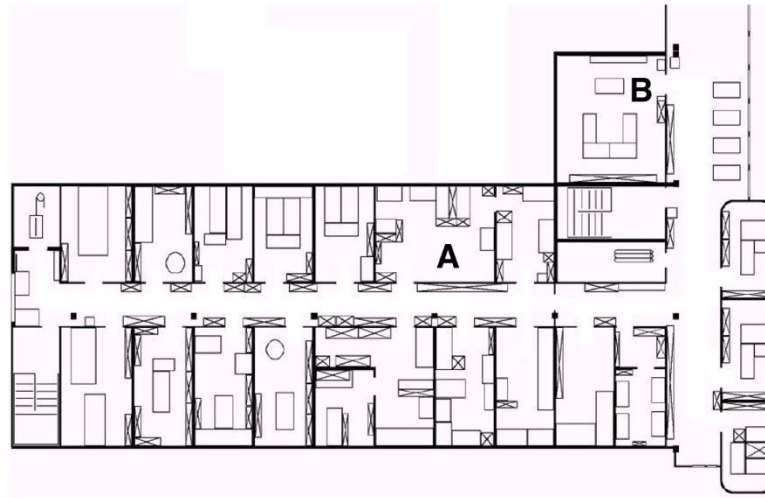
Navigation includes ...

- Mobile robots navigation includes different interrelated activities:
 - i. perception, as obtaining and interpreting sensory information;
 - ii. exploration, as the strategy that guides the robot to select the next direction to go;
 - iii. mapping, involving the construction of a spatial representation by using the sensory information perceived;
 - iv. localization, as the strategy to estimate the robot position within the spatial map;
 - v. path planning, as the strategy to find a path towards a goal location being optimal or not; and
 - vi. path execution, where motor actions are determined and adapted to environmental changes.





Should you localize?



- Question: Navigate from A to B
 - explicit localization with reference to a map
 - Designing sets of behaviors that together result in the desired robot motion (behavior-based)
 - Follow left wall till the goal is reached (Bug Algorithm)
 - But where is the goal?





Sensors for Localization

- Contact sensors: Bumpers
- Odometry
 - Wheel Encoders
- Internal sensors
 - Accelerometers (spring-mounted masses)
 - Gyroscopes (spinning mass, laser light)
 - Compasses, inclinometers (earth magnetic field, gravity)
- Proximity sensors
 - Sonar (time of flight)
 - Radar (phase and frequency)
 - Laser range-finders (triangulation, tof, phase)
 - Infrared (intensity)
- Visual sensors: Cameras
- Satellite-based sensors: GPS





Issues in Localization

- Sensor and effector uncertainty is responsible for the difficulties of localization.
 - Sensor noise
 - Inaccuracy and incompleteness
 - non-uniqueness of sensors readings, or sensor aliasing
 - mobile robot effectors introduce uncertainty about future state.
- All of these un-modeled sources of error result in inaccuracy between the physical motion of the robot, the intended motion of the robot and the proprioceptive/internal sensor estimates of motion.





Probabilistic Robotics

- Why Probabilistic approaches?
- Probabilistic approaches are more robust to any kind of uncertainty
- Current Probabilistic algorithms are efficient.
- The key idea in probabilistic robotics is to represent uncertainty using probability theory: instead of giving a single best estimate of the current robot configuration, probabilistic robotics represents the robot configuration as a probability distribution over all possible robot poses. This probability is called belief.





Probability

- Let X denote a random variable. Let P denote the probability that X takes on a specific value x : $P(X = x)$.
- If X takes on discrete values, we say that X is a discrete variable.
- If X takes on continuous values, we say that X is a continuous variable.
- we use $P(x1 < X < x2)$ for continuous spaces
- for continuous spaces we define the probability density function (pdf) $p(x)$ in the following manner:

$$P(x1 \leq X \leq x2) = \int_{x1}^{x2} p(x) dx$$





Conditional Probability and Bayes Rule

- Given a reading z of the sensor, what is the probability that we are at position x_i ?
 - conditional probability $p(x_i | z)$
 - $p(x_i | z) = p(z | x_i) p(x_i)$
 - current probability $p(x_i)$ that we are at x_i , and $p(z | x_i)$, the conditional probability that the sensor reads z if we are in fact at x_i .
- probability that both z and x_i occur, called the joint probability of the two events:
 - $p(z \cap x_i) = p(z | x_i) p(x_i)$
 - Commutative
 - $p(x_i | z) p(z) = p(x_i \cap z) = p(z \cap x_i) = p(z | x_i) p(x_i)$





Bayes Rule

$$p(x_i | z) = p(z | x_i) p(x_i) / p(z)$$

Belief = likelihood . prior / evidence

- If we know $p(z | x_i)$ and $p(x_i)$ for each i ,
- $p(z)$, the total probability of the event z , can be computed by summing the individual known probabilities:

$$p(z) = \sum_i p(z | x_i) p(x_i) \text{ [Discrete]}$$

$$p(z) = \int_i p(z | x_i) p(x_i) \text{ [Continuous]}$$





Problem

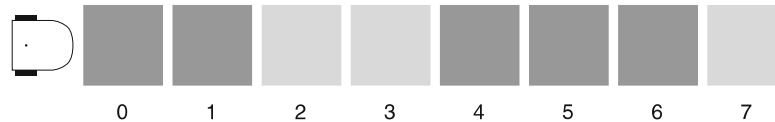
- Imagine a robot is placed in a 1D hallway divided into 8 discrete cells, numbered 0 through 7.
- The robot doesn't know its exact position initially but can move and sense its surroundings.
- The robot:
 - Has a probability of starting in any cell.
 - Can move left or right.
 - Uses a distance sensor that can detect walls or open spaces.
- This problem requires the robot to *estimate* its location (cell) by combining:
 1. Prior knowledge of its position (initially unknown).
 2. Sensor readings as it moves along the hallway.
- When the robot moves (say, to the right), we use a transition model to update the belief (update the position of the robot)





Bayes Problem

- Consider a robot that is navigating within a known environment for which it has a map.



- Let x_i be the event that we are at position i and let z be the event that the robot detects a door. Initially, $p(x_i) = 0.125$ for all positions x_i , and, if the robot is at a door, the probability that the sensor detects this correctly is 0.9, while the probability that it incorrectly detects a door is $1 - 0.9 = 0.1$. The probability of detecting a door, $p(z)$, is obtained by summing the probabilities at each position, where the probability is $0.125 \times 0.9 = 0.1125$ at a position with a door and $0.125 \times 0.1 = 0.0125$ at a position with no door:

$$p(z) = 0.1125 + 0.1125 + 0.0125 + 0.0125 + 0.1125 + 0.1125 + 0.1125 + 0.0125 = 0.575$$





Bayes Problem

- Bayes rule, the probability of being at position i with a door if a door is detected is:

$$p(x_i | z) = p(z | x_i) p(x_i) / p(z) = 0.9 \times 0.125 / 0.575 = 0.196$$

- while the probability of being at position i with no door, but incorrectly a door is detected is:

$$p(x_i | z) = p(z | x_i) p(x_i) / p(z) = 0.1 \times 0.125 / 0.575 = 0.022$$





Normalization

- the sum over all positions i of the probability that the robot is at position i was shown above to be:

$$p(z) = 0.1125 + 0.1125 + 0.0125 + 0.0125 + 0.1125 + 0.1125 + 0.1125 + 0.0125 = 0.575$$

- The probabilities must be normalized by dividing by the sum 0.575 so that the sum will be 1. The normalized probabilities are $0.1125/0.575 \approx 0.19$ and $0.0125/0.575 \approx 0.02$, which do add up to 1:

$$p(z) = 0.19 + 0.19 + 0.02 + 0.02 + 0.19 + 0.19 + 0.19 + 0.02 \approx 1$$

- the robot is trying to localize itself by using a sensor to detect doors. Based on its sensor readings, it updates its probability of being at different positions using the Bayesian framework. This helps the robot to more accurately estimate its position over time, especially in environments with repeated patterns like doors.





Statistics

- mean μ of a set of values $\{x_1, \dots, x_n\}$ is

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i .$$

- Variance is a measure of the spread of a set of values.

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2 .$$

- covariance $\text{cov}(x_1, x_2)$ is the connection between the datasets x_1 and x_2

$$\text{cov}(x_1, x_2) = \frac{1}{n-1} \sum_{i=1}^n (x_{1,i} - \mu_1)(x_{2,i} - \mu_2) .$$

- covariance matrix combines the variances and the covariances:

$$\begin{bmatrix} s^2(x_1) & \text{cov}(x_1, x_2) \\ \text{cov}(x_2, x_1) & s^2(x_2) \end{bmatrix} .$$





Bayes Rule

$$P(x, y) = P(x | y)P(y) = P(y | x)P(x)$$

$\Rightarrow$

$$P(x | y) = \frac{P(y | x) P(x)}{P(y)} = \frac{\text{likelihood} \cdot \text{prior}}{\text{evidence}}$$

$$P(x | y) = \frac{P(y | x) P(x)}{P(y)} = \eta P(y | x) P(x)$$

$$\eta = P(y)^{-1} = \frac{1}{\sum_x P(y | x) P(x)}$$

Law of total probability:

$$P(x) = \int P(x, z) dz$$

$$P(x) = \int P(x | z) P(z) dz$$

$$P(x | y) = \int P(x | y, z) P(z | y) dz$$

Bayes Rule with Background Knowledge

$$P(x | y, z) = \frac{P(y | x, z) P(x | z)}{P(y | z)}$$





Bayes filter

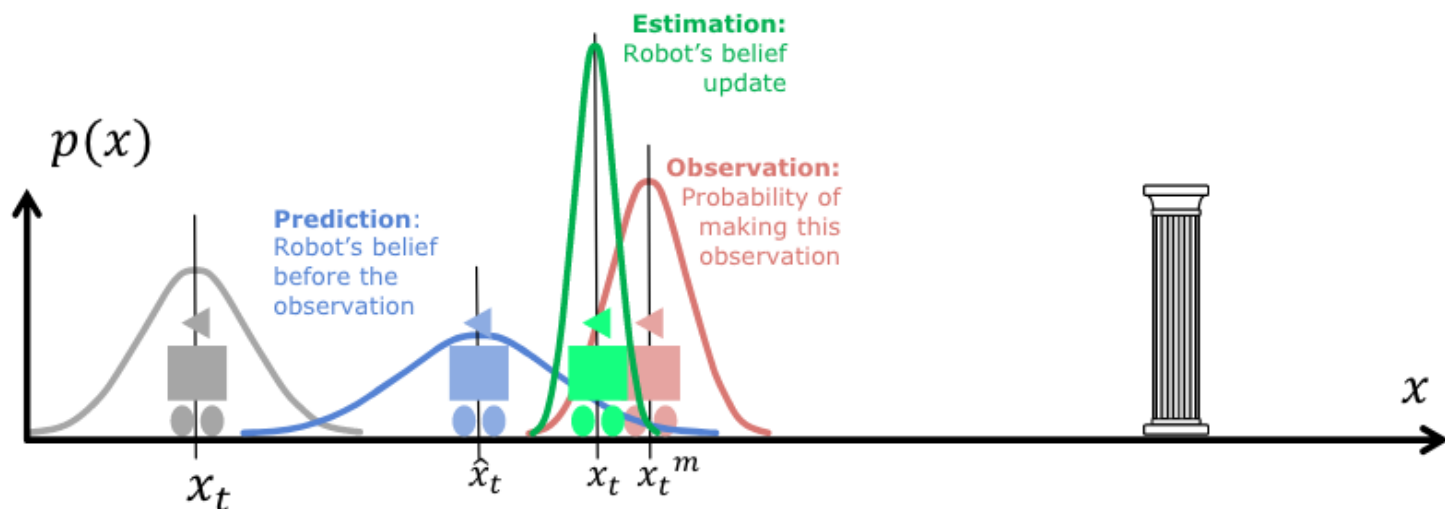
- Bayes filter is a general probabilistic method for estimating unknown probability.
- It is a recursive approach that uses the latest measurement and former estimation to estimate current state.
- Recursive Bayes Filter is an effective approach to estimate the belief about the state of a dynamical system.
- Bayes filter has two essential steps which are the prediction step, and the measurement updating step.
- If the variables are normally distributed and the transitions are linear, the **Bayes filter becomes equal to the Kalman filter**. Non-linear case use particle filter algorithm.





Kalman Filter Localization

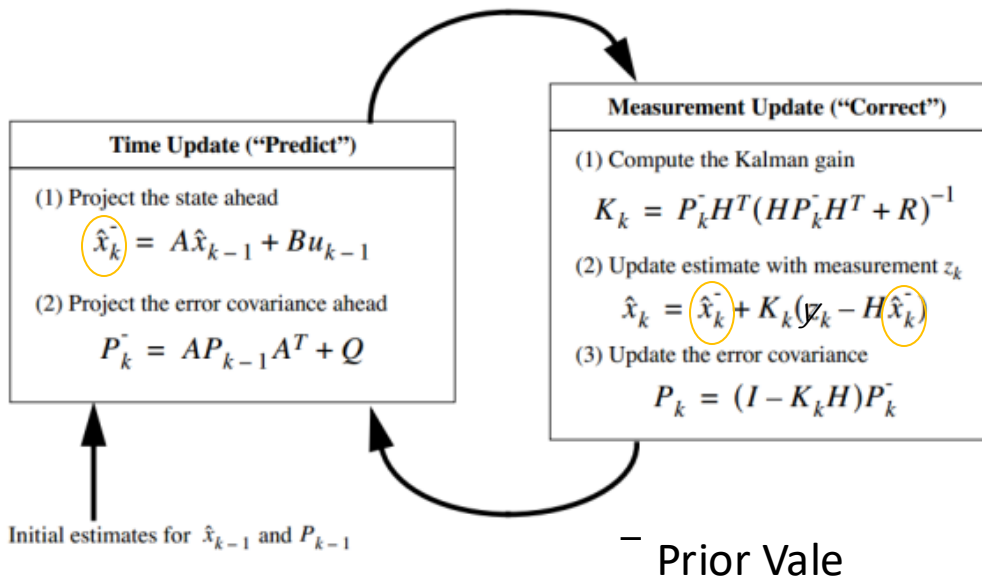
- Kalman filter provides the optimal estimate of the system state, vehicle pose/configuration, assuming that the sensor noise/uncertainty is zero-mean and Gaussian.
- The filter is a recursive algorithm that updates, at each time step, the optimal estimate of the unknown true configuration and the uncertainty associated with that estimate based on the previous estimate and noisy measurement data.
- The Kalman filter is formulated for linear systems but our model of the vehicle's motion is nonlinear – hence we use extended Kalman filter (EKF).





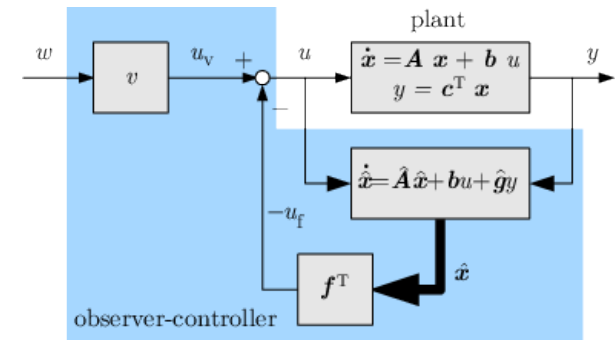
What is Kalman Filter?

- Given the linear dynamical system with state vector $x()$ and output vector $y()$, Kalman Filter provides the “best” estimate of the state vector x .



$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$y_t = C_t x_t + \delta_t$$



Kalman Filter =
State Observer (SC)

- x is system state we want to estimate.
- P is estimation error covariance.
- A (State Transition Matrix) represent the relationship between current system state and last system state. It maps the last system state to current system state.
- B represent the relationship between control process and current system states. It maps the control process to system state.
- Q is control process noise covariance.
- z is the measurement.
- H (measurement function), it maps system state to measurement.
- R is measurement noise covariance.

State observer

$$\hat{x}_{k+1} = A\hat{x}_k + Bu_k + K(y_k - C\hat{x}_k)$$

Kalman filter

$$\hat{x}_k = A\hat{x}_{k-1} + Bu_k + K_k(y_k - C(A\hat{x}_{k-1} + Bu_k))$$





KF

Algorithm Kalman_filter($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

Prediction ①
$$\begin{cases} \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \\ \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \\ K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \end{cases}$$

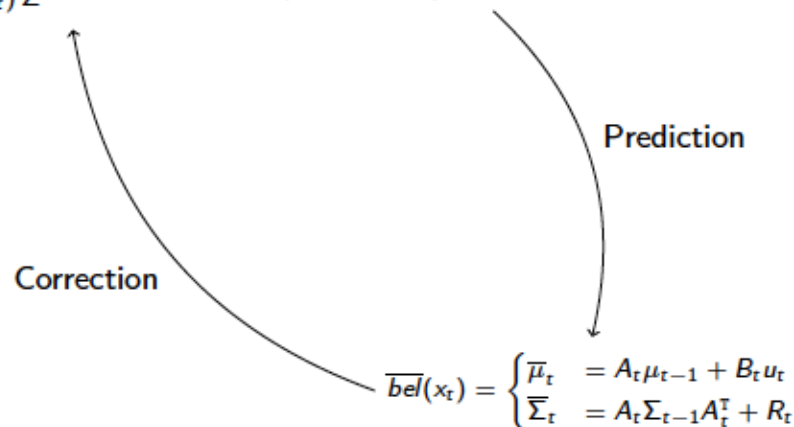
Update ②
$$\begin{cases} \mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \end{cases}$$

return μ_t, Σ_t

$$bel(x_t) = \begin{cases} \mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \end{cases}, K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$$

General KF Filter Cycle

- 1.State prediction
- 2.Measurement prediction
- 3.Measurement
- 4.Data association
- 5.Update





Extended KF

KF

STATE TRANSITION

$$\mathbf{x}_t = \mathbf{A}_t \mathbf{x}_{t-1} + \mathbf{B}_t \mathbf{u}_t + \epsilon_t$$

MEASUREMENT PROBABILITY

$$z_t = \mathbf{C}_t \mathbf{x}_t + \delta_t$$

PREDICTION STEP

$$\begin{aligned}\bar{\mu}_t &= \mathbf{A}_t \mu_{t-1} + \mathbf{B}_t \mathbf{u}_t \\ \bar{\Sigma}_t &= \mathbf{A}_t \Sigma_{t-1} \mathbf{A}_t^T + \mathbf{R}_t\end{aligned}$$

UPDATE STEP

$$\begin{aligned}\mathbf{K}_t &= \bar{\Sigma}_t \mathbf{C}_t^T (\mathbf{C}_t \bar{\Sigma}_t \mathbf{C}_t^T + \mathbf{Q}_t)^{-1} \\ \mu_t &= \bar{\mu}_t - \mathbf{K}_t (z_t - \mathbf{C}_t \bar{\mu}_t) \\ \Sigma_t &= (\mathbf{I} - \mathbf{K}_t \mathbf{C}_t) \bar{\Sigma}_t\end{aligned}$$

EKF

Non-linear

STATE TRANSITION

$$\mathbf{x}_t = g(\mathbf{x}_{t-1}, \mathbf{u}_t, \epsilon_t)$$

MEASUREMENT PROBABILITY

$$z_t = h(\mathbf{x}_t, \delta_t)$$

PREDICTION STEP

$$\begin{aligned}\bar{\mu}_t &= g(\mu_{t-1}, \mathbf{u}_t, 0) \\ \bar{\Sigma}_t &= \mathbf{G}_t \Sigma_{t-1} \mathbf{G}_t^T + \mathbf{N}_t \mathbf{R}_t \mathbf{N}_t^T\end{aligned}$$

UPDATE STEP

$$\begin{aligned}\mathbf{K}_t &= \bar{\Sigma}_t \mathbf{H}_t^T (\mathbf{H}_t \bar{\Sigma}_t \mathbf{H}_t^T + \mathbf{M}_t \mathbf{Q}_t \mathbf{M}_t^T)^{-1} \\ \mu_t &= \bar{\mu}_t + \mathbf{K}_t (z_t - h(\bar{\mu}_t, 0)) \\ \Sigma_t &= (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \bar{\Sigma}_t\end{aligned}$$





EKF

Extended Kalman filter($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

$$\bar{\mu}_t = g(u_t, \mu_{t-1})$$
$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$$

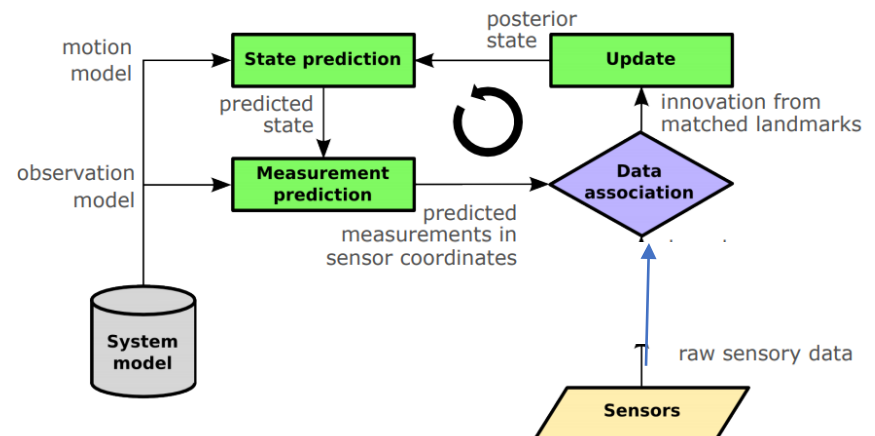
$$K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$$

$$\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$$

$$\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$$

return μ_t, Σ_t

Tracking cycle – Kalman filter



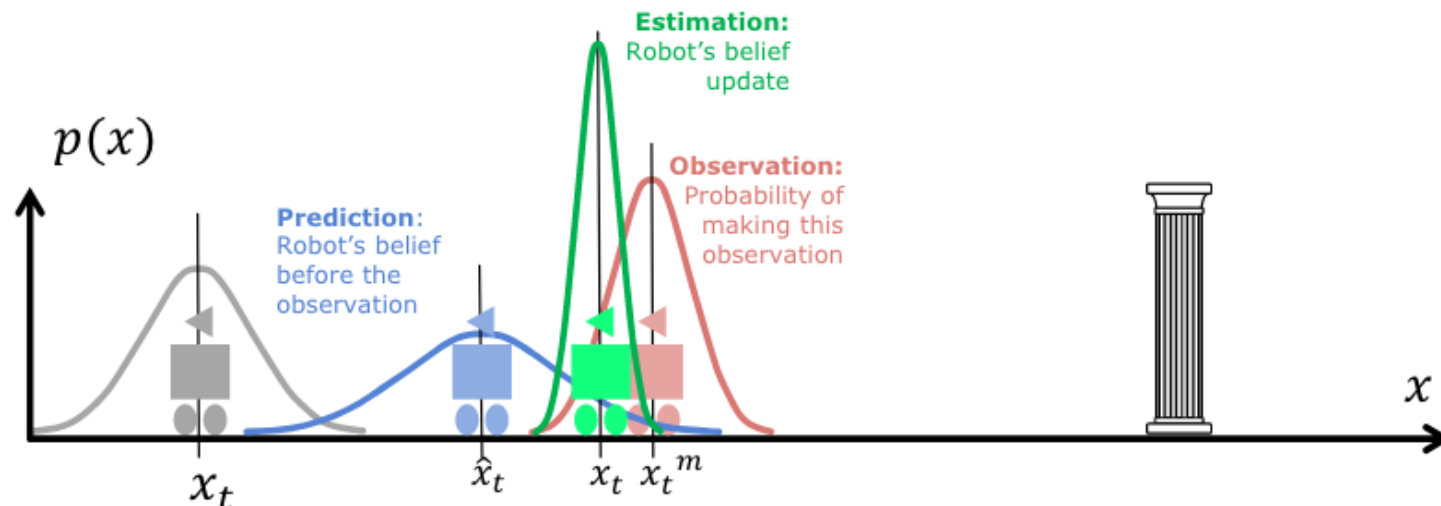
EKF SLAM: Filter Cycle

1. State prediction
2. Measurement prediction
3. Measurement
4. Data association
5. Update



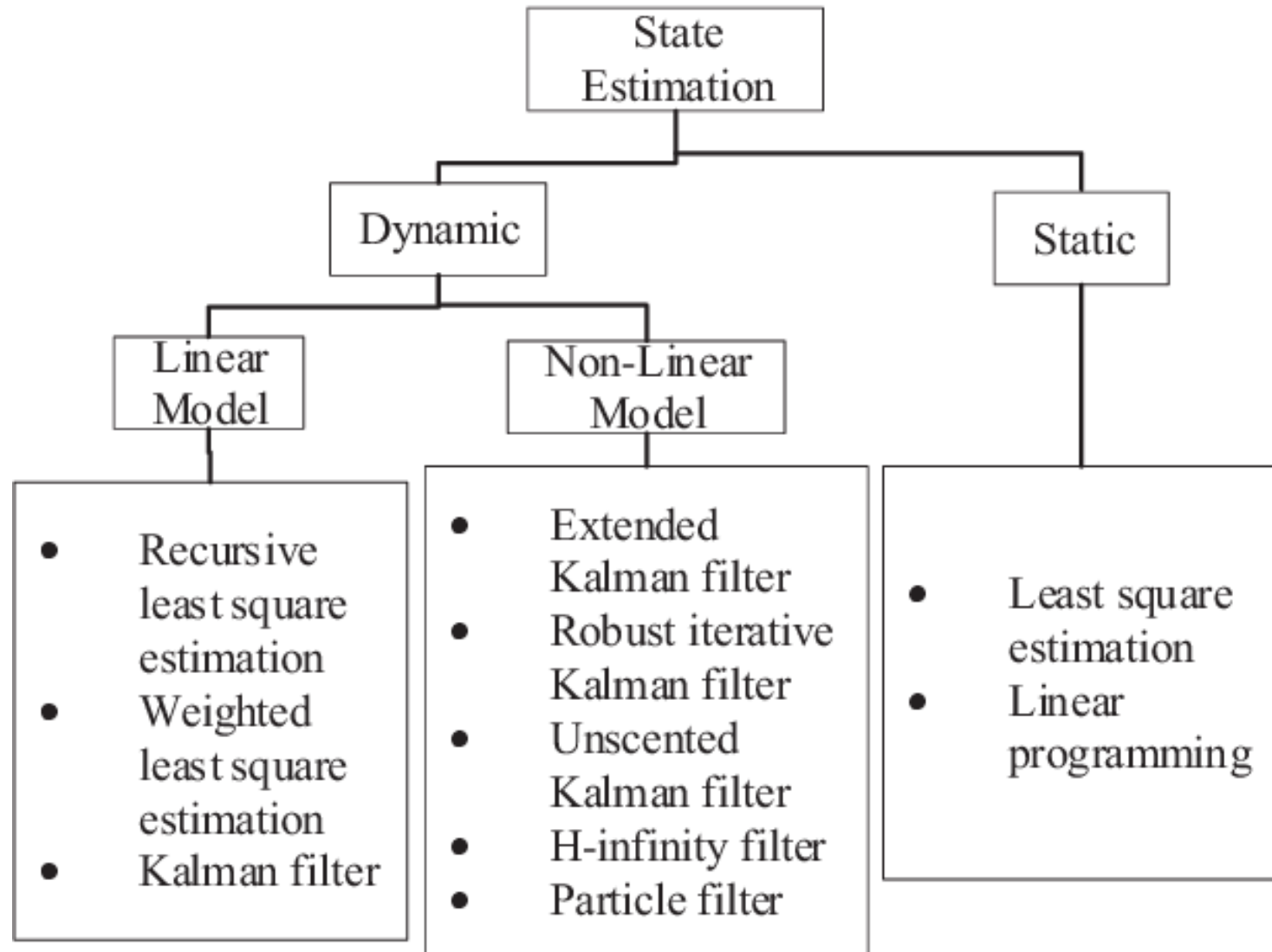
Kalman Filter Localization

- Kalman filter localization tracks the robot's belief state $p(x_t)$ typically as a single hypothesis with normal distribution
- Kalman localization thus addresses the position tracking problem, but not the global localization
- Kalman Filter Algorithm
 - Prediction (ACT) based on previous estimate and odometry
 - Observation (SEE) with on-board sensors
 - Measurement prediction based on prediction and map
 - Matching of observation and map
 - Estimation based on position update (posteriori position)





Other Filters





Localization in ROS

- Robot Localization
 - Pose Estimation
 - http://wiki.ros.org/robot_pose_ekf
 - robot_localization
 - http://wiki.ros.org/robot_localization
 - Adaptive Monte Carlo localization (amcl)
 - <http://wiki.ros.org/amcl>
 - Particle-filter algorithms
 - [mrpt_localization](http://wiki.ros.org/mrpt_localization) (particle-filter algorithms)
 - http://wiki.ros.org/mrpt_localization





Particle Filtering/MCL

- Particle = lot of guess of the location of the robot
- Particle filters are a way to efficiently represent an arbitrary (non-Gaussian) distribution.
- Monte Carlo localization (MCL), also known as particle filter localization, is an algorithm for robots to localize using a particle filter.
- Given a map of the environment, the algorithm estimates the position and orientation of a robot as it moves and senses the environment.





AMCL

- Monte Carlo methods, are a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results.
- adaptive (or KLD-sampling) Monte Carlo localization approach, which uses a particle filter to track the pose of a robot against a known map.
- KLD-sampling method is to bound the approximation error introduced by the sample-based representation of the particle filter.





Demos

